

FP2D_QLRF_NL

User Manual

2D Fokker–Planck solver for ion velocity distribution functions in cylindrical velocity space

F. Louche
LPP-ERM/KMS

Version 2.3 – July 2026

Contents

1	Introduction	3
2	Physical Model	3
2.1	The Fokker–Planck Equation	3
2.2	Coulomb Collision Operator	3
2.2.1	Coulomb Logarithm	4
2.3	Quasi-Linear RF Operator	4
2.4	Self-Collision Operator	4
2.5	Beam Source and Particle Losses	5
3	Numerical Methods	5
3.1	Velocity-Space Grid	5
3.2	Finite-Difference Discretization	6
3.2.1	Boundary Conditions	6
3.3	Advection Stabilization at High $v_{\perp}$ (i_upwind)	6
3.4	Sparse Matrix Storage	7
3.5	Direct Solver: Intel MKL PARDISO	7
4	Algorithms	7
4.1	Steady-State Solver	7
4.2	Time-Dependent Solver – Linear Problem (timefp_7pt)	8
4.2.1	Time Discretization	8
4.2.2	Algorithm	8
4.3	Time-Dependent Solver – Nonlinear Self-Collisions (timefp_7pt_nl)	8
4.3.1	Phi-Distance Kernel Precomputation	8
4.3.2	Time-Stepping Scheme	9
4.3.3	Algorithm	9
4.4	Multi-Phase Time-Stepping	9
4.5	Initial Condition for Time-Dependent Runs	10
4.6	Steady-State Convergence Detection	10
4.7	Diagnostics and Output	10
5	Namelist Reference	11
5.1	General and Output Control	11
5.2	Velocity-Space Grid	11

5.3	Plasma Parameters	12
5.4	Beam Source	12
5.5	RF Heating	13
5.6	Time Integration and Solver Control	13
5.7	Steady-State Convergence Detection	14
6	Output Files	15
6.1	Final Distribution	15
6.2	1D Cuts of the VDF	15
6.3	Time Traces	15
6.4	Ringling / Operator Diagnostics (<code>i_ring_diag</code>)	15
6.5	End-of-Run Balance Diagnostics (<code>stdout</code>)	16
7	Example Namelist	16
8	Compilation and Execution	17
8.1	Build	17
8.2	Running	17
8.3	Plotting	17

1 Introduction

FP2D_QLRF_NL solves the two-dimensional Fokker–Planck (FP) equation for the velocity distribution function (VDF) $f(v_\perp, v_\parallel, t)$ of a minority ion species in a magnetised plasma. The two velocity coordinates are the speed perpendicular to the magnetic field $v_\perp \geq 0$ and the speed parallel to the field $v_\parallel \in (-\infty, +\infty)$. Azimuthal symmetry about the magnetic field direction is assumed throughout.

The code accounts for three physical operators:

1. **Coulomb collisions** with one or more Maxwellian background species (electrons and bulk ions).
2. **Quasi-linear radio-frequency (RF) heating** via the quasi-linear diffusion operator, evaluated using complex Bessel functions.
3. **Self-collisions** (collisions of minority ions with themselves), with five levels of approximation selectable at runtime.

An optional NBI **beam source** term and an associated particle **loss term** are also included.

The code can compute either a *steady-state* solution (direct solve) or a *time-dependent* evolution (implicit time integration). Spatial discretization uses a 7-point Fornberg finite-difference stencil. The resulting sparse linear systems are solved with Intel MKL PARDISO.

2 Physical Model

2.1 The Fokker–Planck Equation

The VDF $f = f(v_\perp, v_\parallel)$ evolves according to

$$\frac{\partial f}{\partial t} = A f + B \frac{\partial f}{\partial v_\perp} + C \frac{\partial f}{\partial v_\parallel} + D \frac{\partial^2 f}{\partial v_\perp^2} + E \frac{\partial^2 f}{\partial v_\perp \partial v_\parallel} + F \frac{\partial^2 f}{\partial v_\parallel^2} + S(v_\perp, v_\parallel), \quad (1)$$

where the six coefficient arrays A – F (called `all00`, `all10`, `all01`, `all20`, `all11`, `all02` in the code) are assembled as the sum of three independently computed contributions:

$$X = X_{\text{coll}} + X_{\text{RF}} + X_{\text{SC}}, \quad X \in \{A, B, C, D, E, F\}, \quad (2)$$

for Coulomb collisions, RF heating, and self-collisions, respectively. The source term S collects the NBI beam injection and the particle loss.

In steady-state mode ($\partial f / \partial t = 0$) the right-hand side of (1) is set to zero and the resulting linear system is solved directly.

2.2 Coulomb Collision Operator

The Coulomb collision operator is the linearized Fokker–Planck operator describing the interaction of the minority ion species a (charge Z_a , mass m_a , thermal velocity v_a) with a Maxwellian background species b (charge Z_b , mass m_b , thermal velocity v_{Tb} , density n_b). The coefficient arrays (`colin00`, ..., `colin02`) for each background species are computed by the subroutine `cblin` and summed over all `nbulk` background species.

The collision frequency scale factor is

$$\Gamma_b = \frac{n_b Z_a^2 Z_b^2 e^4 \ln \Lambda_{ab}}{4\pi \varepsilon_0^2 m_a^2}, \quad (3)$$

where $\ln \Lambda_{ab}$ is the Coulomb logarithm evaluated at each time step (see Section 2.2.1).

2.2.1 Coulomb Logarithm

Two distinct formulas are used:

Ion–electron:

$$\ln \Lambda_{ae} = 24 - \ln \left(\frac{n_e^{1/2}}{T_e} \right), \quad (4)$$

with T_e in eV and n_e in m^{-3} .

Ion–ion:

$$\ln \Lambda_{ab} = 23 - \ln \left[Z_a^2 Z_b^2 \frac{m_a + m_b}{m_a T_b + m_b T_a} \left(\frac{n_a Z_a^2}{T_a} + \frac{n_b Z_b^2}{T_b} \right)^{1/2} \right]. \quad (5)$$

In time-dependent runs the Coulomb logarithm is recomputed at each time step using the current effective temperature of the minority ion species.

2.3 Quasi-Linear RF Operator

When radio-frequency heating is active (`irf` = -1), the quasi-linear diffusion operator is added to the coefficient arrays. The RF operator describes the resonant power exchange between the minority ions and an electromagnetic wave characterised by:

- Wave frequency $\omega = 2\pi f$ (`freak` in Hz).
- Parallel wave vector $k_{\parallel}$ (`kpar` in rad/m).
- Perpendicular wave vector $k_{\perp}$ (complex, `kperp` in rad/m).
- Left- and right-hand polarisation components E_+ , E_- (`eplus`, `emin`).
- Static magnetic field B_0 (`b0` in T).
- Harmonic number n (`nharm`).

The quasi-linear diffusion coefficients are computed in the subroutine `qlrfterm` using complex Bessel functions $J_n(k_{\perp} v_{\perp} / \omega_c)$ and their derivatives, evaluated at each grid point. The resonance condition is $\omega - n \omega_c - k_{\parallel} v_{\parallel} = 0$, broadened by a Gaussian of width `delta_RF` in $v_{\parallel}$.

2.4 Self-Collision Operator

Four options are available for the treatment of self-collisions, selected by the flag `isc`:

isc = 0 – No self-collisions. The self-collision coefficients are zero. The FP equation is linear.

isc = 1 – Maxwellian background at fixed temperature. The self-collision operator is computed using the Rosenbluth potentials for a Maxwellian distribution at the Stix effective temperature T_{eff} (computed once at the start of the simulation and kept constant). This adds a fixed linear operator to the coefficient arrays.

isc = 2 – Maxwellian background at varying effective temperature. Same as `isc` = 1, but the background temperature and Coulomb logarithm are updated at each time step to track the evolving effective temperature $T_{\text{eff}}(t) = (2T_{\perp} + T_{\parallel})/3$. Requires a time-dependent run.

isc = 3 – Maxwellian background at the density-characteristic temperature. Same as **isc = 2**, but the background temperature is the *density-characteristic* (cold-bulk) temperature $T_n(t)$ instead of the energy-weighted $T_{\text{eff}}(t)$. T_n is obtained from a phase-space-weighted least-squares fit of $\ln f$ versus v^2 restricted to the dense thermal core — only cells with $f > \text{core_frac} \cdot \max f$ enter the fit — which returns the temperature exactly for a Maxwellian. Being tied to the cold bulk rather than to the supra-thermal tail, this background better represents the population that dominates the collisional drag. The core fraction **core_frac** (default 3.8×10^{-3}) is tunable; the appropriate value depends on the shape of the distribution, so it may need re-calibrating against the fully nonlinear operator for a new scenario. Requires a time-dependent run.

isc = -1 – Fully nonlinear self-collisions. The complete nonlinear Landau/Fokker–Planck self-collision operator

$$\left(\frac{\partial f}{\partial t}\right)_{\text{SC}} = \Gamma_{aa} \frac{\partial}{\partial v_i} \left[H_{ij}[f] \frac{\partial f}{\partial v_j} - G_i[f] f \right], \quad (6)$$

is used, where H_{ij} and G_i are the Rosenbluth potentials built from the *current* distribution $f(t)$. The Rosenbluth potentials are evaluated via a precomputed phi-distance kernel **sum_phi** using Gauss–Legendre quadrature (subroutine **distance_v_gauss_legendre**). This kernel depends only on the velocity grid and is computed once and saved to disk (**sum_phi.dat**); it can be reloaded for subsequent runs on the same grid. The operator is rebuilt at every time step, making the problem nonlinear. Requires a time-dependent run.

2.5 Beam Source and Particle Losses

When **isource** = -1, a neutral-beam injection (NBI) source is added:

$$S(v_{\perp}, v_{\parallel}) = \frac{n_0}{\tau_s} \frac{\delta_G(v_{\perp} - v_{\perp}^{\text{beam}}) \delta_G(v_{\parallel} - v_{\parallel}^{\text{beam}})}{\mathcal{N}}, \quad (7)$$

where $\delta_G(x) = e^{-(x/\sigma)^2}/(\sqrt{\pi}\sigma)$ is a Gaussian approximation to the Dirac delta, $v_{\perp}^{\text{beam}} = v_{\text{beam}} \sin \alpha$, $v_{\parallel}^{\text{beam}} = v_{\text{beam}} \cos \alpha$ with α the injection angle, and $\mathcal{N}$ is the normalization factor

$$\mathcal{N} = \sqrt{\pi} \sigma_{\perp} e^{-(v_{\perp}^{\text{beam}}/\sigma_{\perp})^2} + \pi v_{\perp}^{\text{beam}} \text{erfc}(-v_{\perp}^{\text{beam}}/\sigma_{\perp}), \quad (8)$$

which reduces to $2\pi v_{\perp}^{\text{beam}}$ for $v_{\perp}^{\text{beam}} \gg \sigma_{\perp}$ and to $\sqrt{\pi} \sigma_{\perp}$ for a purely parallel beam ($v_{\perp}^{\text{beam}} = 0$).

The loss term is $-f/\tau_s$ (isotropic particle losses with slowing-down time τ_s).

3 Numerical Methods

3.1 Velocity-Space Grid

The computational domain is $[v_{\perp, \text{min}}, v_{\perp, \text{max}}] \times [v_{\parallel, \text{min}}, v_{\parallel, \text{max}}]$, discretized on a $N_{\perp} \times N_{\parallel}$ grid with 1-based indices (i, j) where $i \in [1, N_{\perp}]$ and $j \in [1, N_{\parallel}]$.

Three grid spacing modes are available, controlled by **ising**:

ising = 0 Uniform spacing in both directions. $\Delta v_{\perp} = (v_{\perp, \text{max}} - v_{\perp, \text{min}})/(N_{\perp} - 1)$, $\Delta v_{\parallel} = (v_{\parallel, \text{max}} - v_{\parallel, \text{min}})/(N_{\parallel} - 1)$.

ising = -1 Two-domain grid in $v_{\perp}$. A finer mesh of **nsing** points is used for $v_{\perp} \leq \text{vboud}$, and a coarser uniform mesh for $v_{\perp} > \text{vboud}$. This provides higher resolution near the magnetic axis where gradients are typically steeper.

ising = +1 Quadratic spacing in $v_\perp$, providing higher resolution near the axis $v_\perp = 0$. The **fd_stencil_2d** routine uses the Fornberg algorithm (**fornberg_weights**) to compute exact finite-difference weights for the local non-uniform spacing, so this mode is fully supported.

The cylindrical Jacobian is $J = 2\pi v_\perp$, so that integrals over velocity space are $\iint f J dv_\perp dv_\parallel$.

The global 1D degree-of-freedom (DOF) index is

$$\text{ix}(i, j) = (i - 1) N_\parallel + j, \quad i \in [1, N_\perp], j \in [1, N_\parallel], \quad (9)$$

giving $N_{\text{dof}} = N_\perp \times N_\parallel$ total DOFs.

3.2 Finite-Difference Discretization

Spatial derivatives are discretized using the **7-point Fornberg stencil** (subroutine **fd_stencil_2d**), which is a fourth-order accurate scheme. The stencil uses 4 points in each direction: at the current node and its 3 neighbours. For a non-uniform $v_\perp$ grid, the Fornberg algorithm computes the exact derivative weights for the local grid spacing via subroutine **fornberg_weights**.

The stencil returns at most 49 entries per row (7 points in $v_\perp$ by 7 in $v_\parallel$), although boundary rows have fewer entries.

3.2.1 Boundary Conditions

Axis ($i = 1, v_\perp = v_{\perp, \text{min}}$): Neumann condition $\partial f / \partial v_\perp = 0$, enforcing axial symmetry.

Outer boundary ($i = N_\perp$): Dirichlet condition $f = 0$.

Parallel boundaries ($j = 1$ and $j = N_\parallel$): Dirichlet condition $f = 0$.

3.3 Advection Stabilization at High $v_\perp$ (**i_upwind**)

The perpendicular drag coefficient B (collisional friction) grows with $v_\perp$ while the perpendicular diffusion D falls off on the supra-thermal tail, so the cell-Péclet number

$$\text{Pe}_\perp = \frac{|B| \Delta v_\perp}{D} \quad (10)$$

increases steeply toward the outer boundary and can reach $\text{Pe}_\perp \sim 40$ at $v_\perp = v_{\perp, \text{max}}$ for high-energy (e.g. second-harmonic ICRF) scenarios. Beyond $\text{Pe}_\perp \gtrsim 2$ a central discretization of the advection term is no longer dissipative; at the outer Dirichlet wall, where the Fornberg stencil is one-sided, the effective perpendicular diffusion of the discrete operator can even become *negative*. A modified-equation analysis of the Dirichlet-truncated wall row gives, in the uniform-spacing limit,

$$D_{\text{eff}} = \frac{223}{360} D - \frac{1}{12} |B| \Delta v_\perp, \quad (11)$$

i.e. the discrete diffusion turns negative exactly when the wall-cell Péclet number exceeds $223/30 \approx 7.4$ (the deleted wall node biases the drag stencil downwind; high order shrinks the classic two-point downwind coefficient from $\frac{1}{2}$ to $\frac{1}{12}$ but cannot change its sign). The assembled operator $\mathcal{L}$ then acquires a small set of eigenvalues with positive real part, localized in a thin layer against the $v_\perp = v_{\perp, \text{max}}$ boundary. Under Crank–Nicolson (**icn** = -1) these modes are amplified and the run eventually blows up, whereas the fully-implicit scheme damps them. The instability is governed by $v_{\perp, \text{max}}$ (the wall velocity) and is essentially independent of the RF term and of the grid resolution.

Setting **i_upwind** = -1 cures this at the discretization level: in every cell with $\text{Pe}_\perp > 2$ the drag term $B \partial f / \partial v_\perp$ is evaluated with a first-order *upwind* one-sided difference (forward for $B > 0$,

backward for $B < 0$, i.e. the side that makes the semi-discrete eigenvalue dissipative), while the diffusion term keeps its central Fornberg weights. In the modified equation the upwind drag moment $+|B| \Delta v_{\perp}/2$ replaces $-|B| \Delta v_{\perp}/12$, so $D_{\text{eff}} > 0$ at *any* Péclet number; the $\text{Pe}_{\perp} > 2$ switching threshold is therefore conservative with respect to the ≈ 7.4 instability threshold above. The sparsity pattern is unchanged (the diffusion stencil already fills every same-column entry), so the phased-PARDISO machinery is untouched. This removes all positive-real-part modes with a bulk-physics change of at most a few percent, because the upwinding acts only on the near-empty tail. The default `i_upwind = 0` keeps the pure central scheme; running fully implicit (`icn = 0`) is an alternative stabilization. Note that first-order upwinding adds some numerical diffusion on the tail; a flux-limited or exponentially-fitted variant would reduce this if higher tail accuracy is needed.

3.4 Sparse Matrix Storage

All matrices are stored in **1-based Compressed Sparse Row (CSR)** format. For a matrix of size N_{dof} :

- `ia(1:Ndof+1)`: row pointer array.
- `ja(1:nnz)`: column indices.
- `aa(1:nnz)`: non-zero values.

Entries within each row are sorted by column index before passing to PARDISO. The maximum number of non-zeros per row is 49 (7×7 stencil), so `nnz_max = 49 Ndof`.

3.5 Direct Solver: Intel MKL PARDISO

All linear systems are solved with **Intel MKL PARDISO** using matrix type `mtype = 11` (real non-symmetric general). The module `pardiso_solver` exposes four phased entry points that exploit PARDISO's factorization phases:

Entry point	PARDISO phases
<code>pardiso_solve_steady</code>	11 + 22 + 33 (full solve, then release)
<code>pardiso_solve_init</code>	11 + 22 (symbolic + numeric factorization)
<code>pardiso_solve_step</code>	33 only (back-substitution)
<code>pardiso_solve_finalize</code>	release internal memory

This phased approach is critical for performance in the time-dependent solver: when the LHS matrix is constant (linear problem), the factorization is done once and only the back-substitution is repeated at each time step.

4 Algorithms

4.1 Steady-State Solver

When `ntimes(1) = 0`, the code solves the steady-state FP equation

$$\mathcal{L}[f] = -S, \tag{12}$$

where $\mathcal{L}$ is the FP differential operator assembled from the coefficient arrays `all00`, ..., `all02`. The two-pass assembly (first count non-zeros, then fill values) builds the CSR matrix, which is then passed to `pardiso_solve_steady` for a direct solve (subroutine `FP_steady_state` in module `mod_linear`).

4.2 Time-Dependent Solver – Linear Problem (`timefp_7pt`)

Used when `ntimes(1) > 0` and `isc ≠ -1`.

4.2.1 Time Discretization

A θ -method is used:

$$(I - \theta \Delta t \mathcal{L}) f^{n+1} = (I + (1 - \theta) \Delta t \mathcal{L}) f^n + \Delta t S. \quad (13)$$

icn	θ	Scheme
-1	0.5	Crank–Nicolson
+1	0.75	Intermediate
else	1.0	Fully implicit

4.2.2 Algorithm

1. **Assemble** the FP operator $\mathcal{L}$ in CSR format (two-pass loop).
2. **Build LHS:** $M_{\text{lhs}} = I - \theta \Delta t \mathcal{L}$. The sparsity pattern is identical to $\mathcal{L}$ since the diagonal already exists.
3. **Factorize** M_{lhs} once (PARDISO phases 11+22).
4. **Time loop** for $n = 1, \dots, N_{\text{times}}$:
 - (a) Recompute the Coulomb logarithm and, for `isc` $\in \{1, 2, 3\}$, the Maxwellian self-collision background (at the fixed Stix, $T_{\text{eff}}(t)$, or $T_n(t)$ temperature, respectively) from the current distribution, then rebuild the FP operator (allowing a time-varying Coulomb logarithm and self-collision term).
 - (b) Rebuild M_{lhs} with the updated operator.
 - (c) Compute the RHS: $b = (I + (1 - \theta) \Delta t \mathcal{L}) f^n + \Delta t S$ via a hand-coded sparse matrix–vector product.
 - (d) Enforce zero RHS on boundary DOFs.
 - (e) Solve $M_{\text{lhs}} f^{n+1} = b$ (PARDISO phase 33 only).
 - (f) Compute diagnostics (density, energy, power, momentum).
5. **Finalize** PARDISO.
6. If `isource = 0`: rescale $f \leftarrow f \cdot n_0 / \int f d^3v$ to conserve total particle number.

4.3 Time-Dependent Solver – Nonlinear Self-Collisions (`timefp_7pt_n1`)

Used when `ntimes(1) > 0` and `isc = -1`. The LHS matrix now changes at every time step because the self-collision operator depends on f^n .

4.3.1 Phi-Distance Kernel Precomputation

Before entering the time loop, the phi-distance kernel $\Phi_{ij} = \phi(|\mathbf{v}_i - \mathbf{v}_j|)$ (Gauss–Legendre quadrature, $O(N_{\text{dof}}^2)$) is cached on disk in a per-case file `sum_phi-CASENAME.dat` (or `sum_phi.dat` when `casename` is empty). Since the kernel is grid-dependent, the file header stores a grid signature (`nperp`, `npar`, `vperp_min/max`, `vpar_min/max`). Caching is fully self-managing — no user flag is required:

Cache file state	Action
Missing	Compute kernel and save
Present, signature matches	Load kernel (skip the quadrature)
Present, signature differs or truncated	Recompute and overwrite

Because the signature is verified automatically, a stale kernel from a different velocity grid can never be silently reused.

4.3.2 Time-Stepping Scheme

The θ parameter has an additional option for the NL solver:

icn	θ	Comment
-1	0.5	Crank–Nicolson (may be unstable at high $v_{\perp}$; see Sec. 3.3)
+1	0.75	Intermediate
else	1.0	Fully implicit (recommended)

4.3.3 Algorithm

1. **Build sparsity pattern** of $\mathcal{L}$ using the linear coefficients only (pattern is fixed; NL SC uses the same stencil).
2. **Factorize symbolically** (PARDISO phase 11 only).
3. **Time loop** for $n = 1, \dots, N_{\text{times}}$:
 - (a) **Build NL self-collision term**: call `main_nlterm( $f^n$ )` to compute the self-collision coefficient arrays `sc**` from the current distribution via the phi-distance kernel.
 - (b) **Update Coulomb logarithm** and rebuild the linear FP coefficients.
 - (c) **Assemble total operator**: $\mathcal{L} = \mathcal{L}_{\text{lin}} + \mathcal{L}_{\text{SC}}$.
 - (d) **Rebuild LHS**: $M_{\text{lhs}} = I - \theta \Delta t \mathcal{L}$.
 - (e) **Compute RHS**: $b = (I + (1 - \theta) \Delta t \mathcal{L}) f^n + \Delta t S$.
 - (f) **Solve** (PARDISO phases 22+33 every step since M_{lhs} changes).
 - (g) Compute diagnostics.
4. **Finalize** PARDISO.

Note on start-up: when starting from $f = 0$ with a beam source (i.e., `istart = 0`), the NL self-collision term is skipped for the first few steps until the beam density becomes significant enough for the Rosenbluth potentials to be numerically meaningful.

4.4 Multi-Phase Time-Stepping

The namelist parameters `ntimes` and `timestep` are each 3-element arrays, enabling up to three sequential phases with different time steps:

- Phase k runs `ntimes(k)` steps of duration `timestep(k)` seconds.
- Phases with `ntimes(k) = 0` are skipped.
- Phases 2 and 3 append to the output files (restart in time).

Old single-phase input files (`ntimes = N`, `timestep = dt`) remain backward-compatible: the scalar value fills element 1 and elements 2–3 default to zero.

4.5 Initial Condition for Time-Dependent Runs

The initial distribution f^0 is selected by `istart`:

<code>istart</code>	Initial condition
0	$f^0 = 0$ everywhere. Requires <code>isource</code> = -1.
1	Stix Maxwellian solution (default).
2	Steady-state solution of the linear solver without self-collisions.
3	Steady-state solution of the linear solver with Maxwellian SC background.

4.6 Steady-State Convergence Detection

When `i_ss_check` = -1, the code monitors the relative *rate of change* of four indicators over a rolling window of `n_ss_window` steps: the total kinetic energy $\Delta E/E$, the perpendicular kinetic energy $\Delta E_\perp/E_\perp$ (which equilibrates last under RF drive and is the most discriminating), the RF power $\Delta P_{\text{RF}}/P_{\text{drive}}$, and the net power $\Delta P/P_{\text{drive}}$. Each relative change is divided by the *physical duration* of the window ($t_{\text{now}} - t_{\text{now}-W}$), so the criterion measures a relative change *per second*. When all four rates fall below `ss_tol`, the run stops early with a “CONVERGED” message.

Because the window duration is taken from the simulation time rather than a step count, `ss_tol` is *independent of the time step*: two runs with different `timestep` that are equally close to steady state report the same rate, so the same `ss_tol` enforces the same physical stringency (the duration is also measured correctly across phase boundaries where `timestep` changes). **Units:** `ss_tol` is a per-second relative rate; e.g. `ss_tol` = 3.d-4 stops the run when all four indicators change by less than 3×10^{-4} per second of simulated time.

4.7 Diagnostics and Output

At each time step the following quantities are written to disk:

- Density $n(t) = \int f J d^3v$.
- Total, perpendicular, and parallel kinetic energies.
- Effective temperature $T_{\text{eff}} = (2T_\perp + T_\parallel)/3$, where $T_\perp = \frac{1}{2}m\langle v_\perp^2 \rangle$ and $T_\parallel = m\langle v_\parallel^2 \rangle$.
- Density-characteristic temperature T_n (the cold-bulk temperature from the $\ln f$ -vs- v^2 log-slope fit; it is the background used by `isc` = 3 but is written as a diagnostic for *all* runs).
- Near-axis and whole-grid minima of f (a negative- f monitor: small negative undershoots near the axis can corrupt low-velocity moments such as T_n).
- Perpendicular anisotropy factor = $100 T_\perp/(T_\perp + T_\parallel)$ (%), ranging from 0% (all-parallel) to 50% (Maxwellian) to 100% (all-perpendicular).
- Power densities (collisional by species, RF, beam source, losses, self-collision total), and their perpendicular/parallel split for self-collisions.
- Momentum transfer rates (perpendicular and parallel) for each operator.
- Coulomb logarithm(s) vs. time.

At the end of a run, balance diagnostics (power, density, and optionally momentum) are printed to standard output.

5 Namelist Reference

The code reads its input from standard input (`stdin`) as a Fortran NAMELIST group named `INPUT`. A typical invocation is:

```
FP2D_QLRF_NL.exe < jet_beam_case1.dat
```

The namelist file may omit any variable that has a default value; the code will use the default.

The parameters are grouped thematically below.

5.1 General and Output Control

Parameter	Default	Description
<code>casename</code>	"	Character string (max. 64 chars) appended to every output filename and plot title, e.g. 'JET-beam7-TD0-NLSC'. Leave empty for unnamed runs.
<code>iplot_pow</code>	-1	Controls writing of power-vs-time output files. -1: write all power files. 0: skip (useful when only energy and density diagnostics are needed).
<code>iplot_mom</code>	0	Controls writing of momentum-transfer-rate files. -1: write all momentum files. 0: skip (default; momentum files are large).
<code>idiag</code>	0	Selects the end-of-run balance diagnostics printed to stdout. 0: power balance only. -1: all three balances (particle density, momentum, power).
<code>notxt</code>	0	Controls how many <code>.txt</code> output files are written (the <code>.dat</code> restart/kernel files are never affected). 0: write all <code>.txt</code> outputs (normal). 1: suppress all <code>.txt</code> outputs. 2: write only the minimal test/diagnostic set, namely <code>Ekin_perp</code> , <code>Ekin_perp_at_vpar0</code> , <code>fout_at_vpar0</code> , <code>Teff_vs_time</code> , and <code>anisotropy_vs_time</code> . Suppressed files are routed to the OS null device, so the run is otherwise unchanged (all stdout diagnostics still print).
<code>i_ring_diag</code>	0	Per-step ringing / operator diagnostics for the time-dependent solvers (Section 3.3). 0: off. -1: dump the assembled operator $\mathcal{L}$ (<code>L_matrix.txt</code> , COO format) and $\delta f = f^{n+1} - f^n$ snapshots (<code>ring_df_*.txt</code>) for off-line eigenvalue / stability analysis in <i>both</i> time-dependent solvers; the NL solver additionally writes a per-step scalar trace (<code>ring_diag.txt</code>). Diagnostic only; the solution is unaffected.

5.2 Velocity-Space Grid

Parameter	Default	Description
<code>nperp</code>	-	Number of grid points in $v_{\perp}$ (perpendicular direction).

Parameter	Default	Description
<code>npar</code>	–	Number of grid points in $v_{\parallel}$ (parallel direction).
<code>vperp_min</code>	–	Minimum perpendicular velocity (m/s). Should be 0 or very small.
<code>vperp_max</code>	–	Maximum perpendicular velocity (m/s). Should be at least $3\text{--}5\times$ the beam or thermal velocity.
<code>vpar_min</code>	–	Minimum parallel velocity (m/s). Must be negative.
<code>vpar_max</code>	–	Maximum parallel velocity (m/s). Must be positive and equal to $ \text{vpar_min} $ for symmetric grids.
<code>ising</code>	0	Grid spacing mode in $v_{\perp}$. 0: uniform. -1 : two-domain (fine for $v_{\perp} \leq \text{vbound}$, coarser above). $+1$: quadratic spacing, higher resolution near $v_{\perp} = 0$.
<code>nsing</code>	–	Number of fine grid points in the inner domain when <code>ising</code> = -1 . Must satisfy $1 \leq \text{nsing} < \text{nperp}$.
<code>vbound</code>	–	Boundary between fine and coarse regions in $v_{\perp}$ when <code>ising</code> = -1 (m/s).

5.3 Plasma Parameters

Parameter	Default	Description
<code>nbulk</code>	–	Number of background species (including electrons). Species 1 is always electrons. Up to 10 species are supported.
<code>t(1:nbulk)</code>	–	Temperatures of background species (eV). <code>t(1)</code> : electron temperature T_e . <code>t(2)</code> : temperature of first ion species, etc.
<code>aa</code>	–	Mass number of the minority (test) ion species.
<code>za</code>	–	Charge number (atomic number) of the minority ion species.
<code>ab(1:9)</code>	–	Mass numbers of background ion species 2, 3, ... (species 1 is always electrons with m_e/m_p).
<code>zb(1:9)</code>	–	Charge numbers of background ion species 2, 3, ...
<code>ne</code>	–	Electron density (m^{-3}).
<code>xpart</code>	–	Concentration of minority ion species relative to n_e , i.e. $n_a = \text{xpart} \times n_e$.
<code>xb(1:9)</code>	–	Concentrations of background ion species 2, 3, ... relative to n_e . Quasi-neutrality is assumed: $\sum_b Z_b n_b = n_e$.

5.4 Beam Source

Parameter	Default	Description
<code>isource</code>	–	Beam source flag. -1 : include NBI beam source and particle losses. 0: no source; distribution is sourceless (total particles conserved).
<code>beam_ekin</code>	–	Beam injection energy (keV).

Parameter	Default	Description
<code>beam_angle_deg</code>	–	Injection angle (degrees) measured from the magnetic field direction. 0°: purely parallel injection. 90°: purely perpendicular injection.
<code>beam_dvperp</code>	–	Width $\sigma_{\perp}$ of the Gaussian approximation to the delta function in $v_{\perp}$ (m/s). A value of $\sim 5\text{--}10\%$ of $v_{\perp}^{\text{beam}}$ is typical; must be resolved by the grid.
<code>beam_dvpar</code>	–	Width $\sigma_{\parallel}$ of the Gaussian in $v_{\parallel}$ (m/s).
<code>taus</code>	–	Slowing-down time τ_s (s). Sets the source amplitude $S_0 = n_a/\tau_s$ and the loss rate.

5.5 RF Heating

Parameter	Default	Description
<code>irf</code>	–	RF flag. –1: include quasi-linear RF operator. 0 or else: no RF.
<code>frek</code>	–	Wave frequency f (Hz).
<code>b0</code>	–	Static magnetic field B_0 (T).
<code>nharm</code>	–	Resonance harmonic number n (e.g. 1 for fundamental, 2 for second harmonic).
<code>kpar</code>	–	Parallel wave vector $k_{\parallel}$ (rad/m).
<code>kperp</code>	–	Perpendicular wave vector $k_{\perp}$ (complex, rad/m).
<code>eplus</code>	–	Left-hand polarisation amplitude E_+ (complex, V/m).
<code>emin</code>	–	Right-hand polarisation amplitude E_- (complex, V/m).
<code>delta_RF</code>	–	Width of the Gaussian broadening of the resonance in $v_{\parallel}$ (m/s). Must be wide enough to resolve the resonance on the grid.

5.6 Time Integration and Solver Control

Parameter	Default	Description
<code>ntimes(1:3)</code>	0,0,0	Number of time steps per phase. <code>ntimes(1) = 0</code> : steady-state run. <code>ntimes(1) > 0</code> : time-dependent run. Up to three phases with different step sizes are supported; phases with <code>ntimes(k) = 0</code> are skipped. Old-format scalar values (<code>ntimes = N</code>) are backward-compatible and fill <code>ntimes(1)</code> .
<code>timestep(1:3)</code>	0,0,0	Time step size per phase (s). <code>timestep(k)</code> is used for phase k . Backward-compatible with scalar input.
<code>icn</code>	–	Time-differencing scheme. –1: Crank–Nicolson ($\theta = 0.5$). +1: intermediate scheme ($\theta = 0.75$). else: fully implicit ($\theta = 1.0$).
<code>iold</code>	–	Initial condition source. 0: fresh start (use <code>istart</code> to select the initial condition). –1: restart from a previous run by reading <code>xout.dat</code> .

Parameter	Default	Description
<code>istart</code>	1	Initial distribution for fresh time-dependent runs (<code>iold</code> = 0). 0: $f^0 = 0$ (beam-driven only; requires <code>isource</code> = -1). 1: Stix Maxwellian. 2: Steady-state solution without self-collisions. 3: Steady-state solution with Maxwellian self-collisions.
<code>isc</code>	–	Self-collision treatment. 0: no self-collisions (linear FP equation). 1: Maxwellian background at fixed Stix temperature. 2: Maxwellian background at evolving effective temperature $T_{\text{eff}}(t)$ (requires TD run). 3: Maxwellian background at the density-characteristic temperature $T_n(t)$ (requires TD run; see <code>core_frac</code>). -1: fully nonlinear self-collision operator (requires TD run).
<code>core_frac</code>	3.8×10^{-3}	Core fraction defining the T_n log-slope fit for <code>isc</code> = 3: only grid cells with $f > \text{core_frac} \cdot \max f$ enter the $\ln f$ -vs- v^2 fit. Smaller values weight the cold bulk more heavily. Ignored unless <code>isc</code> = 3; the appropriate value is scenario-dependent.
<code>i_upwind</code>	0	Advection stabilization of the perpendicular drag term $B \partial f / \partial v_{\perp}$ (Section 3.3). 0: central Fornberg weights (default). -1: Péclet-hybrid upwinding — in cells where the cell-Péclet number $ B \Delta v_{\perp} / D > 2$ the drag term uses a one-sided upwind difference, restoring dissipativity of the advection-dominated outer boundary layer and removing the Crank–Nicolson high- $v_{\perp}$ instability.
<code>i_freeze_lnl</code>	0	Test mode (linear solver <code>timefp_7pt</code> , <code>isc</code> $\neq$ -1). 0: normal (the Coulomb logarithm and Maxwellian self-collision background track the evolving $T_{\text{eff}}(t)$). -1: freeze the Coulomb logarithm at the initial T_{eff} so the operator coefficients no longer depend on f — a truly linear operator — to isolate the temperature-dependent- $\ln \Lambda$ nonlinearity.

5.7 Steady-State Convergence Detection

Parameter	Default	Description
<code>i_ss_check</code>	0	Enable automatic convergence detection. 0: disabled (run for the full <code>ntimes</code> steps). -1: stop early when all convergence criteria are met.
<code>n_ss_window</code>	50	Rolling window width (number of time steps) over which the convergence indicators $\Delta E/E$, $\Delta n/n$, $\Delta P/P$ are evaluated.
<code>ss_tol</code>	10^{-3}	Relative convergence tolerance. All three indicators must fall below this value for the run to stop.

6 Output Files

All output files are written to the working directory. When `casename` is non-empty, the string `-casename` is inserted before the file extension.

6.1 Final Distribution

fout.txt Final VDF as a list of $(v_{\perp}, v_{\parallel}, f)$ triples.

xout.dat Binary restart file: one header line (final time) followed by the packed 1D solution vector. Used with `iold = -1`.

fstix.dat Stix Maxwellian reference distribution.

6.2 1D Cuts of the VDF

fout_at_vpar0.txt, **fout_at_vperp0.txt**, **fout_at_vperpmax.txt**, **fout_at_vparmax.txt**: values of f along the four boundary slices of the domain.

6.3 Time Traces

Written at every time step during the simulation:

density_vs_time.txt Columns: time (s), $n(t)$ (m^{-3}).

energy_vs_time.txt Columns: time, E_{tot} (keV), $E_{\perp}$ (keV).

anisotropy_vs_time.txt Columns: time, perpendicular anisotropy (%).

Teff_vs_time.txt Columns: time, T_{eff} (keV).

Tn_vs_time.txt Columns: time, T_n (keV) – density-characteristic (cold-bulk) temperature; written for all runs.

fmin_axis_vs_time.txt Columns: time, near-axis min f , whole-grid min f – negative- f monitor.

power_coll_tot_vs_time.txt Total collisional power density (MW/m^3).

power_coll_e_vs_time.txt Electron collisional power density.

power_coll_ionN_vs_time.txt Ion N collisional power density.

power_coll_self_vs_time.txt Self-collision power density (4 columns: time, total, $P_{\text{SC}}^{\perp}$, $P_{\text{SC}}^{\parallel}$).

power_NBI_vs_time.txt Beam injection and particle loss power.

power_RF_vs_time.txt RF power density.

coulomb_log_vs_time.txt Coulomb logarithm for background ion species.

coulomb_log_self_vs_time.txt Self-collision Coulomb logarithm.

momentum_coll_*_vs_time.txt Momentum transfer rates (2 columns per file: $\perp$ and $\parallel$ components), written when `iplot_mom = -1`.

6.4 Ringing / Operator Diagnostics (`i_ring_diag`)

When `i_ring_diag = -1`, the time-dependent solvers write additional files for off-line stability analysis (see Section 3.3):

L_matrix.txt The assembled FP operator $\mathcal{L}$ in coordinate (COO) format: a header line **ndof nnz** followed by **row col value** triples (1-based). Written by both time-dependent solvers — the NL solver overwrites it every 50 steps (so the final file is near steady state), the linear solver writes it once on step 1.

ring_df_n.txt $\delta f = f^{n+1} - f^n$ snapshots on step 1 and every 50 steps (columns: $v_\perp$, $v_\parallel$, δf , $D_\perp$, $D_\parallel$).

ring_diag.txt (NL solver only) Per-step scalar trace: step, time, T_{eff} , $\ln \Lambda_{aa}$, checkerboard/Nyquist projection $\sum (-1)^{i+j} f$, min/max of f and their cells, and the cell of largest $|\delta f|$.

These files underpin the eigenvalue diagnosis of the high- $v_\perp$ Crank–Nicolson instability described in Section 3.3.

6.5 End-of-Run Balance Diagnostics (stdout)

A power balance (and, when **idiag** = -1, particle density and momentum balances) is printed to standard output at the end of the run. Values are given in MW/m³ for power, N/m³ for momentum, and s⁻¹ for particle rates.

7 Example Namelist

The following example illustrates a time-dependent D-beam simulation in a D plasma with nonlinear self-collisions, using a two-phase time-stepping strategy: 1000 coarse steps followed by 1000 finer steps. It lists *every* **&INPUT** parameter; the RF block is included for completeness but is only read when **irf**=-1, and the diagnostic/advanced flags are shown at their default (off) values.

```
&INPUT
  casename = 'JET-beam7-TD0-NLSC',

  ! Grid
  nperp = 151, npar = 151,
  vperp_min = 0.d0, vperp_max = 5.d6,
  vpar_min = -4.d6, vpar_max = 4.d6,
  ising = 1, nsing = 25, vbound = 0.07d6,

  ! Plasma: electrons + D ions
  nbulk = 2,
  t = 15.d3, 10.d3, 8*0.d0,
  aa = 2.d0, za = 1.d0,
  ab = 2.d0, 8*0.d0,
  zb = 1.d0, 8*0.d0,
  ne = 5.0d19,
  xpart = 0.05d0,
  xb = 0.50d0, 8*0.d0,

  ! D-NBI beam at 120 keV, 90 degrees
  isource = -1,
  beam_ekin = 120.d0,
  beam_angle_deg = 90.d0,
  beam_dvperp = 0.12d6,
  beam_dvpar = 0.20d6,
  taus = 300.d-3,

  ! RF quasi-linear operator (b0..delta_RF read only when irf = -1)
  irf = 0,
```

```

b0 = 3.45d0,
frek = 53.d6,
nharm = 1,
kpar = 2.d0,
kperp = (30.d0, 1.d0),
eplus = (1.d3, 3.25d3),
emin = (3.d3, 3.75d3),
delta_RF = 0.5d6,

! Time integration: two phases
icn = 1,
ntimes = 1000, 1000, 0,
timestep = 1.d-3, 10.d-3, 0.d0,
iold = 0,
istart = 0,
isc = -1,
core_frac = 3.8d-3,

! Steady-state convergence detection
i_ss_check = -1,
n_ss_window = 50,
ss_tol = 1.d-4,

! Output
iplot_pow = -1,
iplot_mom = -1,
idiag = -1,
notxt = 0,

! Diagnostics / advanced numerics (default 0 = off)
i_ring_diag = 0,
i_freeze_lnl = 0,
i_upwind = 0,
/

```

8 Compilation and Execution

8.1 Build

Open the Visual Studio solution `FP2D_QLRF_NL/FP2D_QLRF_NL.sln` and build the `Release|x64` configuration. The compiler is Intel `ifx` with Intel MKL (sequential). The executable is placed at `FP2D_QLRF_NL/x64/Release/FP2D_QLRF_NL.exe`.

8.2 Running

The program reads from `stdin`:

```
FP2D_QLRF_NL.exe < inputs/jet_beam_case1.dat
```

Output files are written to the working directory.

8.3 Plotting

Use the companion Python script `fp2d_plot.py`:

```
python fp2d_plot.py run FP2D_QLRF_NL.exe inputs/jet_beam_case1.dat --show
python fp2d_plot.py plot x64/Release --casename JET-beam7-TD0-NLSC --save plots/
```

```
python fp2d_plot.py compare x64/Release --cases JET-beam7-TD0-Lin JET-beam7-TD0-NLSC
```

See `fp2d_plot-readme.txt` for the full documentation of the plotting tool.